Estruturação de Dados com Pandas

Processamento JSON da API CoinGecko

Apresentamos o código em Python necessário para extrair dados relevantes da API CoinGecko e estruturá-los em um DataFrame do Pandas.

O script itera sobre a lista dados_crypto, extraindo as seguintes propriedades essenciais:

- Nome da moeda e símbolo em maiúsculas
- Preço atual em USD (current_price)
- Variação percentual de 24h (price_change_percentage_24h)
- Capitalização de mercado e volume total

Esses registros são consolidados na lista crypto_list, gerando o df_crypto. Usamos head() e shape para validação.

```python
# Conversão da lista de dados para Pandas DataFrame
crypto_list = []
for coin in dados_crypto:
crypto_list.append({
'name': coin['name'],
'symbol': coin['symbol'].upper(),
'price_usd': coin['current_price'],
'change_24h_pct': coin['price_change_percentage_24h'],
'market_cap': coin['market_cap'],
'volume_24h': coin['total_volume']
})
df_crypto = pd.DataFrame(crypto_list)
# Validação estrutural do DataFrame
print(df_crypto.head())
print(df_crypto.shape)
```

Consumo de Dados com a API REST Countries

Requisição GET & Filtragem

Utilizamos a API gratuita **REST Countries** para obter dados demográficos e geográficos globais de forma direta.

Fluxo de Execução:

- Realiza uma chamada GET para o endpoint final;
- Converte a resposta recebida diretamente para JSON;
- Itera sobre a lista de países retornada.

Filtramos apenas os países de interesse: **Brasil, Estados Unidos, China, Índia e Alemanha**.

Para cada um, extraímos o nome, população, área (km^2), região e capital (tratando nulos com .get()) para estruturar o DataFrame df_paises.

```python
# Consome API e filtra dados geográficos
import pandas as pd
import requests

url = "https://restcountries.com/v3.1/all"
response = requests.get(url).json()

alvo = ["Brazil", "United States", "China", "India", "Germany"]
dados = []

for pais in response:
    nome = pais['name']["common"]
    if nome in alvo:
        dados.append({
            "País": nome,
            "População": pais.get("population"),
            "Área": pais.get("area"),
            "Região": pais.get("region"),
            "Capital": pais.get("capital", [""])[0]
        })

df_paises = pd.DataFrame(dados)
```

Demonstração de Web Scraping

Extração de Notícias do Hacker News

Para demonstrar o Web Scraping, utilizamos a biblioteca **`BeautifulSoup`** para extrair as principais notícias do portal Hacker News (https://news.ycombinator.com/).

- **Requisição e Parser:** Após fazer a requisição HTTP e obter o HTML, o parser **`lxml`** cria o objeto **`soup`**.
- **Seletores CSS:** Usamos o método **`find_all`** para localizar as tags **`<a>`** com a classe **`titleline`**.
- **Iteração e Consolidação:** Um loop itera sobre os 10 primeiros resultados coletando o título e o link (**`href`**), salvando no **`df_noticias`**.

```python
# Script de Scraping Hacker News
import requests
from bs4 import BeautifulSoup
import pandas as pd

url = "https://news.ycombinator.com/"
response = requests.get(url)
soup = BeautifulSoup(response.text, "lxml")

# Localiza os links com a classe titleline
tags = soup.find_all("a", class_="titleline")
noticias = []

for tag in tags[:10]:
noticias.append({
"titulo": tag.get_text(),
"link": tag.get("href")
})
df_noticias = pd.DataFrame(noticias)
```

Hora de colocar a mão na massa!

Atividade Individual — Próximos 20 minutos

Objetivos & Diretrizes:

- Implemente o código de coleta para o seu tema escolhido.
- Colete no mínimo **10 registros** por API ou Web Scraping.
- Realize a limpeza básica (conversão de tipos e dados ausentes).
- Consolide os resultados em um **DataFrame final**.
- Salve o progresso em um arquivo **CSV** e documente o processo.

Dica: Use os exemplos fornecidos, comece pequeno e salve frequentemente!



Como Lidar com Erros Comuns no Desenvolvimento

ERROS DE API

- **Status 404:** Indica uma URL incorreta; verifique a documentação da API.
- **Status 429:** Significa que você atingiu o limite de requisições (Rate limit); aumente o tempo de pausa com `time.sleep()`.
- **JSONDecodeError:** Geralmente ocorre quando a API retorna uma página HTML de erro em vez de JSON; revise a URL.

ERROS NO SCRAPING

- **KeyError:** Acontece se você tentar acessar uma chave inexistente no JSON; use o método `.get()` para evitar quebras.
- **AttributeError:** Significa que a tag não foi encontrada no HTML.
- **DataFrame vazio:** O loop ou a função `find_all()` podem ter falhado.

Higienização de Dados com Pandas

Após a coleta, a limpeza rápida dos dados é essencial. O processo envolve as seguintes etapas fundamentais:

- **1. Verificar valores ausentes:** Mapear falhas de preenchimento utilizando df.isnull().sum().
- **2. Tratar os ausentes:** Preencher lacunas com zero através de fillna(0) ou eliminar linhas via dropna().
- **3. Converter tipos de dados:** Ajustar colunas numéricas com to_numeric() e padronizar datas com to_datetime().
- **4. Eliminar duplicatas:** Remover registros repetidos redundantes do conjunto com drop_duplicates().
- **5. Renomear colunas:** Padronizar e traduzir rótulos para melhor clareza técnica utilizando rename().
- **6. Filtrar registros:** Definir restrições de consistência para o negócio, mantendo por exemplo apenas preços maiores que zero.

```
# 1. Verificar valores ausentes
print(df.isnull().sum())

# 2. Tratar valores nulos
df['vendas'] = df['vendas'].fillna(0)
df = df.dropna(subset=['id'])

# 3. Converter tipos de dados
df['preco'] = pd.to_numeric(df['preco'],
errors='coerce')
df['data'] = pd.to_datetime(df['data'])

# 4, 5 e 6. Eliminar, Renomear e Filtrar
df = df.drop_duplicates()
df = df.rename(columns={'vnds': 'vendas'})
df = df[df['preco'] > 0]
```

Análise Exploratória Básica

Permite entender a estrutura do seu dataset recém-criado através de funções essenciais do Pandas:

`df.shape`
Exibe o número total de linhas e colunas.

`df.head()`
Mostra as primeiras linhas para verificação visual.

`df.dtypes`
Lista os tipos de dados de cada coluna.

`df.describe()`
Gera estatísticas descritivas das colunas numéricas.

`df.isnull().sum()`
Quantifica valores ausentes por variável.

`df.columns.tolist()`
Retorna a lista com os nomes de todas as colunas.

```python
# Diagnóstico Rápido do Dataset
import pandas as pd
df = pd.read_csv('dataset.csv')

# Executando funções exploratórias
print(df.shape)
print(df.head())
print(df.dtypes)
print(df.describe())
print(df.isnull().sum())
print(df.columns.tolist())
```

Visualizações com matplotlib.pyplot

A criação de visualizações simples ajuda a identificar padrões e correlações nos dados com facilidade.

Gráfico de Barras (`plt.bar`)

Ideal para comparar valores categóricos. Permite configurar o tamanho da figura, cores, títulos, rótulos e a rotação dos eixos para melhor leitura.

Gráfico de Dispersão (`plt.scatter`)

Utilizado para visualizar a relação ou correlação direta entre duas variáveis numéricas contínuas (X e Y).

** Ambas as abordagens utilizam `plt.show()` para renderizar o gráfico na tela.*

```python
# Importação da biblioteca
import matplotlib.pyplot as plt

# Opção 1: Gráfico de Barras
plt.figure(figsize=(8, 5))
plt.bar(categorias, valores, color='blue')
plt.xticks(rotation=45)
plt.show()

# Opção 2: Gráfico de Dispersão
plt.scatter(var_x, var_y, color='red')
plt.show()
```

Etapa Final: Dataset e Documentação

Salvamos o dataset utilizando `df.to_csv('meu_dataset.csv', index=False, encoding='utf-8')` para garantir que o índice não seja exportado.

Em seguida, o script gera o arquivo de documentação `documentacao.md` em Markdown contendo o tema, data, URLs, propósito, dicionário de dados, estatísticas gerais e insights observados.

```
# Exportando o dataset processado sem salvar o índice do Pandas
df.to_csv('meu_dataset.csv', index=False, encoding='utf-8')

# Registrando a documentação em arquivo Markdown (documentacao.md)
with open('documentacao.md', 'w', encoding='utf-8') as f:
f.write("""# Documentação do Projeto\n"""
f"- Tema: Coleta Automatizada\n- Data: 2026-08-12\n"
f"- Propósito, Dicionário de Dados, Estatísticas e Insights\n"
"""[Conteúdo adicional omitido por brevidade...]""" )
```

Checklist e Autoavaliação do Projeto

Antes de finalizar, faça uma verificação completa utilizando o Checklist do Projeto abaixo para confirmar o cumprimento de todas as etapas:

- ☑️ Tema e justificativa definidos
- ☑️ Fontes de dados identificadas
- ☑️ Coleta de dados planejada
- ☑️ Código implementado
- ☑️ Mínimo de 10 registros coletados
- ☑️ Dados limpos (tipos e ausentes)
- ☑️ DataFrame final criado
- ☑️ Análise exploratória realizada
- ☑️ Pelo menos um gráfico gerado
- ☑️ Dataset salvo em formato CSV
- ☑️ Todo o processo documentado
- ☑️ Notebook salvo com sucesso

Autoavaliação do Aluno

10 a 12

Resultado: EXCELENTE

7 a 9

Resultado: MUITO BOM

4 a 6 itens: Exige revisão do projeto

Menos de 4: Busque ajuda com o professor

Chegou a Hora do Show!

Hora do Show: Apresente seu Projeto (10 minutos)

Cada aluno ou dupla terá 1 minuto para apresentar os resultados do seu projeto de forma objetiva e ágil.

Exemplo de Pitch

Coleta de dados de criptomoedas via **CoinGecko API**, destacando o domínio do Bitcoin no market cap e a dificuldade inicial com JSONs aninhados.

A apresentação deve seguir uma estrutura clara:

[1] O tema escolhido e a justificativa

[2] As fontes de dados utilizadas

[3] O método aplicado (API ou Web Scraping)

[4] O volume total de registros coletados

[5] Um insight interessante descoberto durante a análise

[6] A maior dificuldade técnica enfrentada

[7] O que faria de diferente em uma próxima iteração

Atividade: Feedback Construtivo

Os 3 Pilares do Feedback Estruturado

1. O Mais Interessante
Aponte o que achou de mais destacado ou inovador no projeto apresentado.

2. Sugestão Prática
Forneça uma recomendação acionável e realista para aprimorar o trabalho.

3. Pergunta Genuína
Faça um questionamento que demonstre curiosidade real pelo desenvolvimento ou dados.

⏱️ Duração: 5 Minutos

Regras Fundamentais:
- Mantenha sempre o máximo respeito.
- Apresente apenas sugestões construtivas.
- Faça perguntas motivadas por curiosidade real.

Objetivo:
Promover o aprendizado mútuo e elevar a qualidade final de todos os projetos de dados.

Regra de Ouro do Feedback:

"Foque no projeto, não na pessoa."

Ideias de Projetos Avançados para Portfólio

Para inspirar futuros desenvolvimentos de portfólio, apresentamos ideias de projetos mais complexos de Ciência de Dados com os objetivos e métodos de coleta descritos a seguir.



1. Análise de Sentimentos
Coleta de tweets sobre uma marca para classificar a percepção do público.

2. Previsão de Preços
Coleta do histórico de ações financeiras para modelar tendências futuras.

3. Sistema de Recomendação
Utilização de dados extraídos de catálogos de filmes.

4. Mapas de Calor
Mapeamento de ocorrências a partir de dados públicos de criminalidade.

5. Desenvolvimento de Chatbots
Treinamento de modelos de linguagem com base em FAQs raspadas de sites.

Checklist de Boas Práticas

DESENVOLVIMENTO DE CÓDIGO

- Documente o processo detalhadamente desde o início.
- Utilize controle de versionamento com Git e GitHub.
- Teste seus scripts com amostra pequena de dados inicialmente.
- Implemente tratamento de erros robusto (try/except).
- Crie funções modulares e altamente reutilizáveis.
- Adicione comentários claros em trechos de código complexos.
- Compartilhe e publique o resultado final no seu GitHub.

GESTÃO E COLETA DE DADOS

- **Preservação Original:** Salve sempre os dados brutos originais antes de aplicar qualquer função de limpeza.
- **Ética e Limites:** Respeite rigorosamente os rate limits das APIs e adote boas práticas de web scraping.
- **Consistência:** Valide sempre a integridade e consistência dos dados coletados ao final do pipeline.



"Um projeto bem documentado frequentemente tem mais valor do que um código perfeito."

Cinco Armadilhas na Coleta de Dados

Erro de Escopo Amplo

Problema: Tentar coletar tudo de uma vez gera projetos complexos e inacabados.

Solução: Comece pequeno, defina limites realistas e expanda de forma incremental.

Sem Tratamento de Exceções

Problema: Ignorar o tratamento de erros faz o código quebrar e perder todo o progresso.

Solução: Utilize blocos **try/except** para isolar instabilidades e falhas.

1. Sem Documentação

Trabalhar sem registro resulta no esquecimento das etapas. Escreva documentação de forma síncrona à codificação.

2. Ausência de Validação

Resulta na coleta de dados inúteis ou com formatação incorreta. Verifique a qualidade de forma constante.

3. Desistir no Erro

Falhas acontecem. O processo de debugging é uma parte normal, esperada e essencial do desenvolvimento.

Ferramentas Gratuitas e Essenciais

Postman
Fundamental para testar os endpoints das APIs antes de escrever o código em Python.

Google Sheets
Útil para importar arquivos CSV e realizar explorações visuais rápidas do conjunto de dados coletados.

JSON Viewer
Ajuda a visualizar e entender a estrutura de arquivos JSON formatados.

GitHub
Indispensável para o versionamento do código do projeto e compartilhamento público do seu portfólio.

SelectorGadget
Extensão para Chrome que facilita a identificação rápida das tags e classes HTML durante o planejamento do Web Scraping.

Notion / Obsidian
Excelentes opções estruturadas para redigir a documentação técnica detalhada do seu projeto.

Repositórios de Datasets Prontos

Kaggle
`kaggle.com/datasets`

Hospeda mais de 50.000 bases de dados variadas.

dados.gov.br
`dados.gov.br`

Concentra e disponibiliza os dados abertos do governo brasileiro.

UCI Machine Learning Repository
`archive.ics.uci.edu`

Clássico para datasets acadêmicos e testes de algoritmos.

World Bank Data
`dados.bancomundial.org`

Repositório totalmente focado em indicadores de desenvolvimento global.

Google Dataset Search
`datasetsearch.research.google.com`

Um buscador dedicado a encontrar dados em toda a web.

AWS Open Data
`registry.opendata.aws`

Grandes volumes de dados públicos hospedados de forma otimizada na nuvem Amazon.

APIs Gratuitas Populares para Projetos Práticos

OpenWeather
Dados meteorológicos e previsão do tempo.

NewsAPI
Manchetes e artigos de notícias (requer chave gratuita).

Spotify API
Metadados musicais (exige autenticação OAuth).

Twitter API
Extração limitada de tweets para análise.

Reddit API
Coleta de posts e comentários de fóruns públicos.

GitHub API
Métricas de repositórios e atividade de desenvolvedores.

PokéAPI
Informações detalhadas sobre Pokémon e evolução.

REST Countries
Dados demográficos, geográficos e econômicos globais.

CoinGecko
Cotações em tempo real e histórico de criptomoedas.

The Movie DB (TMDb)
Catálogos detalhados de filmes, séries e elenco.

```
Lista colaborativa completa:  github.com/public-apis/public-apis
```

Web Scraping Avançado

Limitações do BeautifulSoup

Quando a biblioteca BeautifulSoup atinge seus limites técnicos, torna-se necessário recorrer a abordagens avançadas para contornar gargalos da web moderna:

- **Sem execução de JavaScript:** Incapaz de processar códigos que estruturam elementos pós-carregamento.
- **Falha em SPAs:** Não consegue raspar sites dinâmicos construídos com frameworks modernos (React, Angular, etc.).
- **Autenticação complexa:** Não possui suporte nativo robusto para lidar com fluxos de login intrincados.

Frameworks Avançados

Para superar esses obstáculos, desenvolvedores utilizam ferramentas projetadas para renderização completa e automação de navegadores:

- **Selenium:** Simula a interação de um navegador real e renderiza códigos JS dinamicamente.
- **Playwright:** Uma alternativa moderna, otimizada e mais rápida ao ecossistema Selenium.
- **Scrapy:** Framework Python completo, altamente escalável e focado em extração massiva.
- **Puppeteer:** Solução de automação equivalente de alto desempenho para o ecossistema Node.js.

Automação e Agendamento de Scripts

Para projetos que exigem a coleta recorrente de dados (como o acompanhamento diário de preços de criptomoedas), a automação é fundamental.

- **Local:** Uso de **Cron jobs** no Linux/Mac ou o **Task Scheduler** no Windows.
- **Nuvem Gratuita:** O **GitHub Actions** permite rodar o código de forma programada e sem custos.
- **Corporativo:** Soluções serverless robustas como **AWS Lambda** e **Google Cloud Functions**.

```python
# Loop de execução contínua com schedule
import schedule
import time
import coletar_dados

# Executa a função todos os dias às 08:00
schedule.every().day.at("08:00").do(coletar_dados)

while True:
schedule.run_pending()
time.sleep(1)
```

Coleta de Dados e Diretrizes da LGPD

Dados Pessoais Protegidos

Qualquer informação que identifique um indivíduo diretamente ou facilite sua identificação.

Exemplos sob a Lei 13.709/2018:

- *Nome, CPF, e-mail, endereço físico*
- *Endereço IP e geolocalização*
- *Histórico de hábitos de consumo*

Recomendação de Segurança:

"Quando houver dúvida sobre a legalidade ou ética do processo, não realize a coleta de forma alguma."

Regras Fundamentais e Exceções

Consentimento Explícito

Não realize coletas sem a autorização clara e finalidade informada ao titular.

Retenção Mínima e Anonimização

Não armazene dados além do prazo do projeto. Anonimize as informações sempre que possível.

Direito de Exclusão

Garanta mecanismos práticos e seguros para a eliminação total dos dados a pedido.

Exceções Previstas

Há permissões com ressalvas estritas para dados que já são manifestamente públicos ou para uso em pesquisas acadêmicas.

Como Divulgar Seu Projeto de Coleta de Dados

01. Código no GitHub

Crie um repositório contendo o script Python, o dataset em CSV e publique opcionalmente o notebook e dataset no **Kaggle**.

→

02. README Crucial

Deve conter obrigatoriamente: Título do projeto, descrição clara do objetivo, instruções de instalação, exemplos de uso, principais resultados e autor.

→

03. Divulgação Ativa

Compartilhe insights do projeto em posts no **LinkedIn** ou escreva artigos técnicos no seu blog pessoal ou no **Medium**.

"No mercado de Ciência de Dados, um projeto bem documentado no GitHub frequentemente tem mais peso do que certificados de cursos."

Próximos Passos na Coleta de Dados

Roteiro prático para continuar evoluindo suas habilidades de engenharia e extração

1. Praticar APIs Diversas

Pratique a extração explorando APIs de temas diferentes dos vistos em aula para expandir seu portfólio.

2. Selenium e Web Scraping

Aprofunde-se em Web Scraping aprendendo o framework Selenium para lidar com sites dinâmicos em JavaScript.

3. Bancos de Dados e SQL

Inicie os estudos em bancos de dados relacionais e SQL para armazenar volumes coletados de forma mais eficiente que em arquivos CSV.

4. APIs REST Avançadas

Estude conceitos avançados de APIs REST, compreendendo métodos de autenticação complexos e paginação de resultados.

5. Competições no Kaggle

Participe ativamente de competições no Kaggle, que frequentemente incluem desafios focados em engenharia e coleta de dados.

6. Contribuição Open Source

Contribua com projetos de código aberto (open source), ajudando a melhorar bibliotecas de scraping mantidas pela comunidade.

> *"A coleta estruturada é a habilidade fundamental que precede qualquer modelo de Machine Learning."*

Resumo das Conquistas

O resumo das conquistas desta aula reflete o fluxo de trabalho real de um Cientista de Dados. Durante a sessão, vocês realizaram com sucesso as seguintes etapas:

(1) Planejaram o escopo de um projeto prático de coleta;

(2) Identificaram e selecionaram fontes de dados viáveis;

(3) Executaram a extração de dados brutos utilizando requisições a APIs ou técnicas de Web Scraping;

(4) Aplicaram rotinas de limpeza para unificar e padronizar os formatos;

(5) Criaram um dataset próprio e exclusivo a partir do zero;

(6) Redigiram a documentação técnica do processo; e

(7) Compartilharam os resultados e insights com a turma.

Marco Alcançado

100%

Prático e Exclusivo

Vocês agora possuem experiência prática comprovada no pilar fundamental da engenharia de dados.

Apliquem este exato fluxo de trabalho nos seus próximos projetos de portfólio.

Checklist Final de Entrega

Antes de encerrarmos, revise o Checklist Final para garantir que todas as entregas da aula foram concluídas. Confirme se você possui:

- [x] Notebook completo com todo o código executável
- [x] Dataset final salvo corretamente em formato CSV
- [x] Documentação do projeto redigida em Markdown
- [x] Mínimo de 10 registros válidos coletados
- [x] Dados limpos e padronizados (tipos corretos e nulos)
- [x] Etapa de análise exploratória executada
- [x] Geração de pelo menos uma visualização gráfica
- [x] Projeto publicado ou pronto para compartilhar (GitHub/Kaggle)
- [x] Participação ativa fornecendo feedback aos colegas
- [x] Registro das principais lições técnicas aprendidas

Se todos os itens estiverem marcados, parabéns por completar um projeto real de coleta de dados de ponta a ponta.

100%

PROJETO CONCLUÍDO

Cronograma das Próximas Aulas

AULAS 5 & 6
Limpeza Avançada

- **Parte 1: Tratamento de valores ausentes, duplicatas e outliers.**
- **Parte 2: Normalização, padronização e variáveis categóricas.**

AULA 7
EDA Avançada

- Análise Exploratória de Dados profunda.
- Criação de visualizações complexas.
- Geração e interpretação de matrizes de correlação.

AULA 8
Modelos de IA

- Introdução à criação de Modelos Preditivos.
- Fundamentos essenciais de classificação.
- Fundamentos essenciais de regressão.

Próximos passos após a Aula 8: Aula 9 (Avaliação de Modelos e hiperparâmetros) e Aula 10 (Deploy prático via API funcional).

** Nota: O conhecimento adquirido em cada sessão é pré-requisito obrigatório e direto para o sucesso da etapa seguinte.*

Fontes de Aprofundamento Recomendadas

Documentação Oficial

Requests: Acesse a documentação oficial da biblioteca em `docs.python-requests.org`

BeautifulSoup: Consulte os recursos completos da biblioteca no site oficial `crummy.com`

Literatura Recomendada

- ***'Web Scraping with Python'*** (Ryan Mitchell): Essencial para consolidação da base teórica de extração de dados.
- ***'Automate the Boring Stuff with Python'*** (Al Sweigart): Excelente guia prático de automação.

Cursos e Aprendizado Prático

Kaggle Learn: Explore os módulos práticos focados especificamente em APIs e Web Scraping.

Real Python: Acesse tutoriais detalhados e guias passo a passo de desenvolvimento.

Recursos em Vídeo: Excelente suporte visual com tutoriais do canal **Corey Schafer** e os bootcamps do **FreeCodeCamp** no YouTube.

Comunidades Ativas

- Participe ativamente resolvendo dúvidas e compartilhando código no famoso **Stack Overflow**.
- Junte-se a discussões globais e fóruns especializados no subreddit **r/webscraping**.
- Interaja em tempo real com desenvolvedores brasileiros no servidor do Discord da comunidade **Data Hackers BR**.

A prática diária é essencial para a fixação do conhecimento. Desenvolva seus próprios scripts!

Sua Opinião é Fundamental

Sua opinião é fundamental para a melhoria contínua do curso. Por favor, reserve 3 minutos para preencher o formulário rápido de feedback acessando o link do Google Forms disponibilizado.

3 Min

TEMPO DE PREENCHIMENTO

Formulário rápido e objetivo focado na avaliação das dinâmicas das aulas.

1. Nível de Dificuldade

Avaliação sobre o nível de dificuldade técnico da aula (de Muito fácil a Difícil).

2. Etapa Mais Útil

Identificação entre Demonstração, Projeto próprio ou Compartilhamento.

3. Fase Desafiadora

Indicação da fase crítica: Planejamento, Coleta, Limpeza ou Documentação.

4. Ritmo de Exposição

Opinião sobre a velocidade de exposição do conteúdo (Lento, Adequado ou Rápido).

5. Nível de Confiança

Escala de 1 a 5 para replicar um projeto de coleta de dados de forma autônoma.

6. Sugestões e Críticas

Espaço de campo aberto para sugestões gerais e críticas construtivas ao curso.

Preparação para a Aula 5: Tarefas de Casa

Para garantir que vocês estejam preparados para a Aula 5, as tarefas de casa exigem a conclusão do ciclo de engenharia de dados iniciado hoje. O foco da próxima aula será exclusivamente a **Limpeza Avançada de Dados**.

1. Finalizar Projeto Prático

Assegurar que as etapas de coleta, limpeza básica e documentação no notebook estejam totalmente completas.

2. Publicar no GitHub

Criar um repositório público contendo todo o código desenvolvido e um arquivo README estruturado e explicativo.

3. Explorar Trabalho dos Colegas

Baixar e analisar de forma prática o dataset gerado por outro aluno da turma para treinar a leitura de dados de terceiros.

4. Pesquisa e Fundamentação Teórica

Iniciar o embasamento conceitual para a próxima aula pesquisando sobre "data cleaning techniques" (técnicas de limpeza de dados).

5. Registro de Dificuldades

Anotar detalhadamente no seu processo quaisquer erros, problemas técnicos ou dúvidas encontradas para discussão coletiva em sala.

Foco da Próxima Aula (Aula 5)
Exclusivamente dedicado à **Limpeza Avançada de Dados** .

Parabéns, Coletores de Dados!

"Dados são o novo petróleo, mas só valem se forem refinados"

Vocês deram um passo técnico gigante hoje, evoluindo da teoria sobre 'como coletar dados' para a execução prática de criar um dataset estruturado totalmente do zero. Todo projeto bem-sucedido de Data Science e Machine Learning começa obrigatoriamente com uma rotina sólida de extração de informações.

Para a próxima aula, tragam o notebook desenvolvido hoje, o dataset CSV coletado e suas dúvidas anotadas. Nosso próximo tema será Limpeza de Dados.